LAPORAN RISET

MODUL 2
"PEMROGRAMAN ROBOTIKA DENGAN KIT IMCLAB MENGGUNAKAN PYTHON"

oleh :

Evan Adwitiya Dwi Listanto / 23081010093

ROBOTIKA A-801
PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
JAWA TIMUR
2026

1. Tujuan

Percobaan ini bertujuan untuk :

1. Merancang program Python menggunakan pustaka pyserial yang mampu mendeteksi port ESP32 secara otomatis (auto-detect) dan membuka koneksi serial.
2. Mengirimkan perintah kontrol (ON, OFF, naik, turun, dan penetapan nilai PWM) dari Python ke ESP32 pada Kit iMCLab.
3. Memahami protokol komunikasi teks berbasis baris (line-based text protocol) antara Python dan ESP32, termasuk mekanisme pengiriman perintah dan pembacaan balasan (feedback).
4. Menganalisis perubahan nilai duty cycle PWM berdasarkan urutan perintah yang dikirimkan.

2. Dasar Teori

1. **Komunikasi Serial dan pyserial.** Pada komunikasi serial (UART), data ditransmisikan secara bit per bit dengan kesepakatan baud rate yang sama pada kedua sisi, yaitu 115200 bps. Keberadaan chip USB-UART pada board ESP32 memungkinkan laptop mengenali perangkat keras tersebut sebagai port COM. Melalui pustaka pyserial, Python dapat berinteraksi dengan port tersebut menggunakan berbagai fungsi bawaan, seperti serial.Serial untuk membuka koneksi, write untuk mengirimkan data, in_waiting untuk memeriksa antrean data yang masuk, serta readline untuk membaca balasan data dengan batas waktu (timeout) tertentu.
2. **Identifikasi Perangkat via VID:PID.** Setiap chip USB memiliki identitas unik berupa pasangan *Vendor ID* (VID) dan *Product ID* (PID). Modul `serial.tools.list_ports` pada Python dapat memindai dan mendaftar seluruh port yang tersedia beserta informasi VID:PID dan deskripsinya. Penggunaan pasangan VID:PID ini jauh lebih andal untuk mendeteksi keberadaan ESP32 dibandingkan bergantung pada nomor port COM,

mengingat penomoran COM dapat berubah secara dinamis saat perangkat dicolokkan ke port USB yang berbeda.

3. **Protokol Perintah Teks Per Baris.** Sistem ini menerapkan protokol komunikasi teks berbasis baris (line-based text protocol), di mana perintah dikirimkan sebagai rangkaian teks yang diakhiri oleh karakter baris baru (newline). Pada sisi penerima (ESP32), karakter yang masuk akan diakumulasikan ke dalam buffer dan baru akan dieksekusi ketika karakter penutup baris terdeteksi. Mekanisme ini memberikan fleksibilitas untuk mengirimkan perintah multi-karakter (seperti PWM:150) maupun perintah tunggal (seperti '1', '0', '+', atau '-').
4. **PWM (Pulse Width Modulation).** Pengaturan kecepatan motor DC dilakukan menggunakan metode PWM. Semakin besar nilai duty cycle yang diberikan (berkisar antara 0 hingga 255 pada resolusi 8-bit), semakin besar pula daya rata-rata yang disalurkan ke motor, sehingga putaran motor akan semakin cepat.

3. Alat, Bahan, dan Konfigurasi Pin

Perangkat Keras yang digunakan dalam Percobaan ini :

- ESP 32
- Modul motor IMCLab
- Kabel USB Data
- Laptop

Perangkat Lunak yang digunakan dalam Percobaan ini :

- Arduino IDE Versi 2.3.10
- Pyhton dengan Jupyther Notebook
- Visual Studi Code
- Versi core ESP32 di Boards Manager : 2.0.17

Fungsi di kode	Variabel di kode	GPIO di kode	Label di papan
LED indikator	ledPin	2	D2
Pin kontrol arah motor 1	motor1Pin1	27	D27
Pin kontrol arah motor 2	motor1Pin2	26	D26
Enable (PWM)	enable1Pin	12	D12

Parameter PWM dan komunikasi pada sketch:

Parameter	Nilai
Frekuensi PWM	30.000 Hz
Kanal dan resolusi PWM	Kanal 0, 8 bit (duty 0 sampai 255)
Duty awal	200
Langkah naik dan turun (DUTY_STEP)	10
Batas duty (DUTY_MIN, DUTY_MAX)	0 dan 255
Baud rate	115200
Timeout pembacaan serial	1 detik

4. Prosedur Percobaan

1. Hubungan Modul ESP 32 Ke Laptop menggunakan kabel USB data.
2. Pastikan Kabel USB yang digunakan mendukung Transfer data, bukan hanya untuk pengisian daya.
3. Periksa Apakah ESP32 sudah terdeteksi pada komputer.
4. Buka aplikasi Arduino IDE.
5. Salin dan tempel Kode berikut ke Arduino IDE.

   Kode `imclab_naik_turun.ino`

```
// ============================================================
// Kontrol Motor DC + LED dengan PWM via Serial
// Perintah:
```

```
//   '1'         -> Motor ON (duty terakhir / default)
//   '0'         -> Motor OFF
//   '+'         -> Naikkan kecepatan (+10)
//   '-'         -> Turunkan kecepatan (-10)
//   'PWM:xxx'  -> Set duty cycle langsung (0-255)
// ============================================================

int ledPin = 2;
char myData = 0;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// Setting PWM properties
const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;
int dutyCycle = 200;          // duty cycle awal
const int DUTY_STEP = 10;     // langkah naik/turun
const int DUTY_MIN  = 0;
const int DUTY_MAX  = 255;

String inputString = "";      // buffer untuk perintah "PWM:xxx"

void setup()
{
  pinMode(ledPin, OUTPUT);
  Serial.begin(115200);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  // konfigurasi PWM
  ledcSetup(pwmChannel, freq, resolution);
  ledcAttachPin(enable1Pin, pwmChannel);

  // pastikan motor mati saat start
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, 0);

  Serial.println("ESP32 Motor Controller siap.");
  Serial.println("Perintah:    1=ON,    0=OFF,    +=naik,    -=turun,
PWM:xxx=set");
}

void applyMotor()
{
  // Terapkan dutyCycle ke driver motor
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, dutyCycle);

  // LED indikator: nyala kalau motor bergerak
  digitalWrite(ledPin, dutyCycle > 0 ? HIGH : LOW);
}

void MotorOn()
{
  // Kalau duty masih 0, pakai default 200
  if (dutyCycle == 0) dutyCycle = 200;
  applyMotor();
  Serial.print("Motor ON, duty = ");
```

```
  Serial.println(dutyCycle);
}

void MotorOff()
{
  dutyCycle = 0;
  applyMotor();
  Serial.println("Motor OFF, duty = 0");
}

void naikkanKecepatan()
{
  dutyCycle += DUTY_STEP;
  if (dutyCycle > DUTY_MAX) dutyCycle = DUTY_MAX;
  applyMotor();
  Serial.print("Kecepatan NAIK, duty = ");
  Serial.println(dutyCycle);
}

void turunkanKecepatan()
{
  dutyCycle -= DUTY_STEP;
  if (dutyCycle < DUTY_MIN) dutyCycle = DUTY_MIN;
  applyMotor();
  Serial.print("Kecepatan TURUN, duty = ");
  Serial.println(dutyCycle);
}

void setDuty(int val)
{
  if (val < DUTY_MIN) val = DUTY_MIN;
  if (val > DUTY_MAX) val = DUTY_MAX;
  dutyCycle = val;
  applyMotor();
  Serial.print("Set duty = ");
  Serial.println(dutyCycle);
}

void loop()
{
  while (Serial.available() > 0)
  {
    char c = Serial.read();

    // Cek perintah "PWM:xxx"
    if (c == '\n' || c == '\r')
    {
      inputString.trim();
      if (inputString.startsWith("PWM:"))
      {
        int val = inputString.substring(4).toInt();
        setDuty(val);
      }
      else if (inputString.length() > 0)
      {
        // perintah satu karakter
        char cmd = inputString.charAt(0);
        if (cmd == '1')      MotorOn();
        else if (cmd == '0') MotorOff();
        else if (cmd == '+') naikkanKecepatan();
        else if (cmd == '-') turunkanKecepatan();
        else {
          Serial.print("Perintah tidak dikenal: ");
          Serial.println(inputString);
        }
```

```
      }
      inputString = "";
    }
    else
    {
      inputString += c;
    }
  }
}
```

6. Pilih board ESP32 melalui menu Tools > Board > esp32 > DOIT ESP32 DEVKIT V1 .
7. Pilih port melalu menu Tools > Port >(Pilih port yang sesuai).
8. Klik Tombol Upload
9. Tunggu hingga proses upload selesai.
10. Buka aplikasi Visual Studio Code**.**
11. Pilih menu File > New File untuk membuat file baru.
12. Simpan file dengan menekan Ctrl + S**.**
13. Pada kolom nama file, tulis nama file dengan ekstensi .inpy,
14. Salin dan tempel kode berikut pada Visual Studio Code.

Kode `Tes_IMCLab_naik_turun.ipynb`

```
# ============================================================
# Program Kontrol Motor ESP32 via Serial (Auto-Detect Port)
# Fitur: ON/OFF + Naik/Turun kecepatan (PWM)
# ============================================================

import serial
import serial.tools.list_ports
import time
import sys

# ============================================
# KONFIGURASI
# ============================================
BAUDRATE = 115200

ESP32_KEYWORDS = [
    "cp210", "cp2102", "cp210x",
    "ch340", "ch341", "ch34x",
    "ftdi", "ft232",
    "usb-serial", "usb serial",
    "usb uart", "uart bridge",
    "silicon labs",
    "espressif", "esp32",
]

ESP32_VID_PID = {
    (0x10C4, 0xEA60), (0x10C4, 0xEA70),
    (0x1A86, 0x7523), (0x1A86, 0x55D4),
    (0x0403, 0x6001), (0x0403, 0x6015),
    (0x303A, 0x1001),
}
```

```
DUTY_STEP = 10   # selisih kenaikan/penurunan di sisi Python
(informatif)


# ============================================
# Auto-detect port ESP32
# ============================================
def list_serial_ports():
    ports = serial.tools.list_ports.comports()
    if not ports:
        print("Tidak ada port serial yang terdeteksi.")
        return []
    print("Port serial yang tersedia:")
    for p in ports:
        vid_pid = ""
        if p.vid is not None and p.pid is not None:
            vid_pid = f" [VID:PID={p.vid:04X}:{p.pid:04X}]"
        print(f"  - {p.device} : {p.description}{vid_pid}")
    return list(ports)


def deteksi_port_esp32(verbose=True):
    ports = serial.tools.list_ports.comports()
    if not ports:
        if verbose:
            print("[!] Tidak ada port serial yang terdeteksi.")
        return None

    kandidat = []
    for p in ports:
        desc = (p.description or "").lower()
        mfg  = (p.manufacturer or "").lower()
        hwid = (p.hwid or "").lower()
        teks = f"{desc} {mfg} {hwid}"

        skor = 0
        if p.vid is not None and p.pid is not None:
            if (p.vid, p.pid) in ESP32_VID_PID:
                skor += 100
        for kw in ESP32_KEYWORDS:
            if kw in teks:
                skor += 50
                break
        if "bluetooth" in teks:
            skor -= 200

        if skor > 0:
            kandidat.append((skor, p))

    if not kandidat:
        non_bt = [p for p in ports
                  if "bluetooth" not in (p.description or
"").lower()]
        if len(non_bt) == 1:
            if verbose:
                print(f"[i] Hanya ada 1 port non-Bluetooth:
{non_bt[0].device}")
            return non_bt[0].device
        if verbose:
            print("[!] Port ESP32 tidak dapat dideteksi otomatis.")
        return None
```

```
    kandidat.sort(key=lambda x: x[0], reverse=True)
    if verbose:
        print("Kandidat port ESP32 (skor tertinggi = paling
mungkin):")
        for skor, p in kandidat:
            print(f"  [{skor:>4}] {p.device} : {p.description}")
    return kandidat[0][1].device


# ============================================
# Koneksi serial
# ============================================
def buka_koneksi(port, baudrate=115200):
    try:
        ser = serial.Serial(port, baudrate, timeout=1)
        time.sleep(2)
        print(f"Terhubung ke ESP32 di {port} @ {baudrate} baud")
        return ser
    except serial.SerialException as e:
        print(f"Gagal membuka port {port}: {e}")
        return None


# ============================================
# Kirim perintah & baca balasan
# ============================================
def kirim_perintah(ser, perintah: str):
    """
    Mengirim perintah ke ESP32 (diakhiri newline).
    Perintah valid: '0', '1', '+', '-', 'PWM:xxx'
    """
    ser.write((perintah + "\n").encode("utf-8"))
    ser.flush()
    time.sleep(0.15)

    # Baca semua baris balasan yang tersedia
    while ser.in_waiting > 0:
        balasan = ser.readline().decode("utf-8",
errors="ignore").strip()
        if balasan:
            print(f"ESP32: {balasan}")
        time.sleep(0.05)


# Wrapper perintah
def motor_on(ser):
    print(">> Motor ON")
    kirim_perintah(ser, "1")

def motor_off(ser):
    print(">> Motor OFF")
    kirim_perintah(ser, "0")

def kecepatan_naik(ser, step=DUTY_STEP):
    """Naikkan kecepatan motor sebanyak `step` (default 10)."""
    print(f">> Kecepatan +{step}")
    kirim_perintah(ser, "+")

def kecepatan_turun(ser, step=DUTY_STEP):
    """Turunkan kecepatan motor sebanyak `step` (default 10)."""
    print(f">> Kecepatan -{step}")
    kirim_perintah(ser, "-")
```

```
def set_pwm(ser, duty: int):
    """Set duty cycle langsung (0-255)."""
    duty = max(0, min(255, int(duty)))
    print(f">> Set PWM = {duty}")
    kirim_perintah(ser, f"PWM:{duty}")


# ============================================
# PROGRAM UTAMA
# ============================================
if __name__ == "__main__":
    print("=" * 50)
    list_serial_ports()
    print("=" * 50)

    PORT = deteksi_port_esp32()
    if PORT is None:
        print("Program dihentikan: port ESP32 tidak ditemukan.")
        sys.exit(1)
    print(f"\n>> Port ESP32 terpilih: {PORT}")

    esp32 = buka_koneksi(PORT, BAUDRATE)

    if esp32 is None:
        print("Koneksi gagal. Periksa kembali PORT dan kabel USB.")
    else:
        try:
            # ---- Demo urutan kontrol motor ----
            print("\n--- Mulai urutan kontrol motor ---")

            motor_on(esp32)
            time.sleep(2)

            # Naikkan kecepatan 3x
            for _ in range(3):
                kecepatan_naik(esp32)
                time.sleep(1)

            # Turunkan kecepatan 2x
            for _ in range(2):
                kecepatan_turun(esp32)
                time.sleep(1)

            # Set PWM langsung
            set_pwm(esp32, 150)
            time.sleep(2)

            motor_off(esp32)
            print("--- Urutan kontrol selesai ---\n")

            # ---- Mode interaktif ----
            print("Mode interaktif:")
            print("  '1'   = Motor ON")
            print("  '0'   = Motor OFF")
            print("  '+'   = Naikkan kecepatan (+10)")
            print("  '-'   = Turunkan kecepatan (-10)")
            print("  'p'   = Set PWM langsung (mis. p150)")
            print("  'q'   = Keluar")

            while True:
                cmd = input("Perintah: ").strip()

                if cmd == "q":
```

```
                    break
                elif cmd == "1":
                    motor_on(esp32)
                elif cmd == "0":
                    motor_off(esp32)
                elif cmd == "+":
                    kecepatan_naik(esp32)
                elif cmd == "-":
                    kecepatan_turun(esp32)
                elif cmd.lower().startswith("p") and
cmd[1:].isdigit():
                    set_pwm(esp32, int(cmd[1:]))
                else:
                    print("Perintah tidak valid! Gunakan 1/0/+/-
/p<angka>/q.")

        except KeyboardInterrupt:
            print("\nDihentikan oleh pengguna.")

        finally:
            try:
                motor_off(esp32)
            except Exception:
                pass
            esp32.close()
            print("Koneksi serial ditutup.")
```

15. Klik Run pada sel pertama untuk menjalankan proses pemilihan port otomatis.

16. Pastikan notebook berhasil mendeteksi port ESP32 yang terhubung.

17. Notebook akan membuka koneksi serial dengan **baud rate 115200**.

18. Notebook menjalankan urutan pengujian motor:

    a. Motor ON (dengan nilai duty default 200).

    b. Jeda selama 2 detik

    c. Kecepatan dinaikkan (+10) niali duty 210.

    d. Jeda selama 1 detik

    e. Kecepatan dinaikkan (+10) niali duty 220.

    f. Jeda selama 1 detik

    g. Kecepatan dinaikkan (+10) niali duty 230.

    h. Jeda selama 1 detik

    i. Kecepatan diturunkan (-10) niali duty 220.

j. Jeda selama 1 detik

k. Kecepatan diturunkan (-10) niali duty 210.

l. Jeda selama 1 detik

m. Kecepatan diturunkan (-10) niali duty 200.

n. Jeda selama 1 detik

o. Mengatur nilai PWM secara langsung ke angka 150.

p. Jeda selama 2 detik

q. Motor Off

19. Pada mode interaktif, ketik 1 untuk ON, 0 untuk OFF, + untuk menambah Kecepatan (+10), - untuk mengurangi Kecepatan (-10), P untuk set kecepatan langsung, dan q untuk keluar.

20. Catat balasan ESP32 yang tampil di notebook dan amati putaran motor

5. Hasil Percobaan

1. Koneksi dan Balasan ESP32

   Saat Menjalankan File `Tes_IMCLab_naik_turun.ipynb` tampil Output sebagai Berikut

```
================================================
Port serial yang tersedia:
  - COM9 : Silicon Labs CP210x USB to UART Bridge (COM9) [VID:PID=10C4:EA60]
  - COM6 : Standard Serial over Bluetooth link (COM6)
  - COM3 : Standard Serial over Bluetooth link (COM3)
  - COM7 : Standard Serial over Bluetooth link (COM7)
  - COM4 : Standard Serial over Bluetooth link (COM4)
  - COM5 : Standard Serial over Bluetooth link (COM5)
  - COM8 : Standard Serial over Bluetooth link (COM8)
================================================
Kandidat port ESP32 (skor tertinggi = paling mungkin):
  [ 150] COM9 : Silicon Labs CP210x USB to UART Bridge (COM9)

>> Port ESP32 terpilih: COM9
Terhubung ke ESP32 di COM9 @ 115200 baud

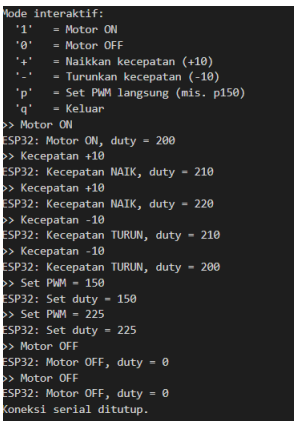
--- Mulai urutan kontrol motor ---
>> Motor ON
ESP32: ESP32 Motor Controller siap.
ESP32: Perintah: 1=ON, 0=OFF, +=naik, -=turun, PWM:xxx=set
ESP32: Motor ON, duty = 200
>> Kecepatan +10
ESP32: Kecepatan NAIK, duty = 210
>> Kecepatan +10
ESP32: Kecepatan NAIK, duty = 220
...
  '+'   = Naikkan kecepatan (+10)
  '-'   = Turunkan kecepatan (-10)
  'p'   = Set PWM langsung (mis. p150)
  'q'   = Keluar
```

Sistem deteksi otomatis port pada program `Tes_IMCLab_naik_turun.ipynb` Berhasil menetapkan port COM9 sebagai port ESP32 dengan skor 150 (terdiri dari 100 poin kecocokan VID:PID dan 50 poin dari kata kunci). Setiap perintah yang dikirimkan ke ESP32 berhasil mendapatkan balasan.

Commented [H1]:

Perintah (Python)	Balasan ESP32	Duty	Jeda
motor_on ('1')	Motor ON, duty = 200	200	2 detik
naik ('+')	Kecepatan NAIK, duty = 210	210	1 detik
naik ('+')	Kecepatan NAIK, duty = 220	220	1 detik
naik ('+')	Kecepatan NAIK, duty = 230	230	1 detik
turun ('-')	Kecepatan TURUN, duty = 220	220	1 detik
turun ('-')	Kecepatan TURUN, duty = 210	210	1 detik
set_pwm(150) ('PWM:150')	Set duty = 150	150	2 detik

Berdasarkan pengujian pada mode interaktif, setiap perintah yang dikirimkan mendapatkan respons langsung dari ESP32.



Diketik	Dikirim	Balasan ESP32	Duty
1	Motor ON	Motor ON	200
+	Kecepatan +10	Kecepatan NAIK, duty = 210	210
+	Kecepatan +10	Kecepatan NAIK, duty = 220	220
-	Kecepatan -10	Kecepatan TURUN, duty = 210	210
-	Kecepatan -10	Kecepatan TURUN, duty = 200	200
p150	Set PWM = 150	Set duty = 150	150
P225	Set PWM = 225	Set duty = 225	225
P125	Set PWM = 125	Set duty = 125	125
0	Motor OFF	Motor OFF	0
q	Motor OFF	Motor OFF, duty = 0; Koneksi serial ditutup.	0

6. Analisis

1. Pesan awal ikut terbaca di perintah pertama. Pada Tabel 5.1, balasan perintah pertama didahului dua baris dari setup() ('ESP32 Motor Controller siap.' dan baris daftar perintah). Pesan itu dikirim saat ESP32 restart, yang terjadi ketika port dibuka dari Python. Meskipun buka_koneksi() menunggu 2 detik, pesan tetap tertampung di buffer, lalu terbaca bersama balasan pertama karena kirim_perintah() membaca semua baris yang tersedia (while ser.in_waiting > 0).

2. Perintah ON ('1') menyalakan motor menggunakan nilai duty terakhir. Berdasarkan kode MotorOn(), nilai duty hanya diatur ke 200 jika sebelumnya bernilai 0. Jika pengguna menekan '+' saat OFF (menjadi 10) lalu menekan '1', motor akan menyala dengan duty 10. Hal ini menyebabkan motor bisa berstatus

ON dengan duty sangat rendah sehingga torsinya tidak cukup untuk memutar motor.

3. Tombol naik menyalakan motor dari keadaan OFF. Perintah '+' pada duty 0 menghasilkan duty 10, dan applyMotor() menyalakan LED karena duty lebih dari nol. Jadi motor dapat dinyalakan hanya dengan '+', tanpa perintah '1'.

4. Satu perintah, satu balasan. Setiap perintah pada Tabel 5.1 dan 5.2 menghasilkan tepat satu baris balasan, membuktikan bahwa protokol per baris berfungsi dengan baik (perintah diproses sekali saat newline tiba). Karena balasan menyertakan nilai duty terbaru, Python dapat memastikan perubahan yang terjadi tanpa harus membaca pin ESP32 secara langsung.

5. Validasi Dua Lapis. Nilai PWM dibatasi di dua tempat: fungsi set_pwm pada Python (0–255) dan fungsi setDuty serta perintah up/down pada ESP32 (menggunakan DUTY_MIN dan DUTY_MAX). Validasi di sisi pengirim mencegah pengiriman data yang salah, sementara validasi di sisi penerima melindungi motor dari perintah eksternal (misalnya dari Serial Monitor). Nilai DUTY_STEP = 10 juga tertulis di kedua sisi, tetapi di Python hanya berfungsi untuk teks tampilan (>> Kecepatan +10). Jika salah satu nilai diubah tanpa sinkronisasi, teks di layar bisa tidak sesuai dengan kenaikan kecepatan yang sebenarnya.

6. Keselamatan saat Program Berakhir. Mekanisme pengamanan diterapkan melalui blok finally yang secara otomatis mengirimkan perintah OFF sebelum menutup port serial. Hal ini dibuktikan pada Tabel 6.2 baris ke-15, di mana saat karakter 'q' diinputkan, ESP32 merespons dengan pesan "Motor OFF, duty = 0" sesaat sebelum pesan "Koneksi serial ditutup." muncul. Dengan mekanisme ini, motor tidak akan tertinggal dalam kondisi menyala meskipun program dihentikan secara normal maupun saat terjadi error.

7. Kesimpulan

1. Program Python berhasil menemukan port ESP32 otomatis (COM11, skor 150) dan membuka koneksi serial 115200 baud.
2. Perintah ON, OFF, naik, turun, dan set PWM berhasil dikirim dari Python, dan ESP32 membalas satu baris untuk tiap perintah dengan nilai duty terbaru
3. Protokol perintah teks per baris memungkinkan perintah satu karakter dan perintah PWM:xxx, dengan validasi nilai di Python dan di ESP32.
4. Duty berubah sesuai perintah dalam langkah 10 pada rentang 0 sampai 255